Assignment 06 - Visitor Pattern: Advanced File System

Course: ShP-2216 Software Design Patterns Student: Dinmukhammed Dauletkhan Group: SE-2538

1. Purpose and design choices

The project is an in-memory hierarchical file system (files and nested directories) with three independent operations: total size, search and statistics. The operations are implemented with the Visitor pattern, so they live outside the element classes and new operations can be added without changing them. Java 17, Maven, JUnit 5.

- Object model: Element interface (getName, accept), abstract class Node (name and parent), FileNode (name, non-negative size) and DirectoryNode (named children, any nesting).
- Double dispatch: FileNode.accept and DirectoryNode.accept both call visitor.visit(this). The first dispatch picks accept by the element type, the second picks the overloaded visit(FileNode) or visit(DirectoryNode) by the static type of this. No instanceof, getClass or casts are used.
- Traversal owner: the visitors. visit(DirectoryNode) loops over getChildren() and calls child.accept(this). Children are kept in a TreeMap, so the order is always by name, and every reachable node is visited exactly once. This keeps accept methods trivial and lets each visitor decide what to do before and after the children (for example, depth and path tracking).
- Paths and depth: SearchVisitor keeps a stack of folder names; the root directory is not pushed, so a path is root-relative and joined with "/". The result is sorted with Collections.sort. StatisticsVisitor keeps currentDepth: the root has depth 0, it increases by one when entering a directory and decreases when leaving; maxDepth is updated for every file and directory.
- State control: all visitors are reusable. The run methods calculate(root), search(root) and collect(root) reset all fields before traversing, so a repeated run never leaks or multiplies old values and the result does not depend on run order.
- Tree integrity: DirectoryNode.add rejects null, a node that already has a parent, a duplicate sibling name and a cycle (walk up from the target directory to see whether the child is one of its ancestors). FileNode rejects a negative size. Invalid changes throw IllegalArgumentException. getChildren returns an unmodifiable copy.
- Search rule: a file matches if its name contains the given text and size >= the minimum size (inclusive).

2. UML diagram

See the last page (UML is excluded from the 3-page limit).

3. Clean Code justifications

Practice	Where	Justification
Meaningful naming	TotalSizeVisitor.calculate, SearchVisitor.search, StatisticsVisitor.collect, DirectoryNode.createsCycle	A name says what the method does, so the code can be read without comments.
Single Responsibility	TotalSizeVisitor, SearchVisitor, StatisticsVisitor; FileNode and DirectoryNode	Each visitor has one operation and the elements only store data and dispatch, so a change in one place does not affect the others.
Small methods	SearchVisitor.matches, enter, leave, buildPath; StatisticsVisitor.updateMaxDepth	Every method does one small step, so visit(...) stays 5 to 7 lines long and is easy to trace.
Encapsulation	DirectoryNode.children (private), getChildren() copy, package-private Node.setParent	Clients can change the tree only through add and remove, so all invariants are always checked.
No type-checking chains	FileNode.accept, DirectoryNode.accept	Double dispatch selects the behavior, so there is no instanceof/if-else, and a new operation is a new class.

4. Sample output (own data, Main)

Demo tree: project/{README.md 1200, src/{Main.java 3400, Utils.java 800, util/{Helper.java 1500, notes.txt 200}}, docs/{guide.txt 5000, images/{logo.png 8000}}, build/ (empty)}.

```
Total size: 20100 bytes
Files: 7
Directories: 6
Max depth: 3
Search 'e' with size >= 1000:
  docs/guide.txt
  src/util/Helper.java
Search '.txt' with size >= 0:
  docs/guide.txt
  src/util/notes.txt
Search 'zzz' with size >= 0:
  (nothing found)
Size of src only: 5900 bytes
Rejected: Duplicate name 'README.md' in 'project'
```

5. Test summary

Command: mvn test (JUnit 5, no manual input). 35 tests in 5 classes:

Test class	Tests	Checks
ElementDispatchTest	3	T1
TotalSizeVisitorTest	5	T2, T6, T7
SearchVisitorTest	10	T3, T4, T6, T7
StatisticsVisitorTest	5	T5, T6, T7
TreeIntegrityTest	12	T8

```
Tests run: 35, Failures: 0, Errors: 0, Skipped: 0
BUILD SUCCESS
```

6. Traversal and extension trade-offs

- Traversal in visitors: the recursion is repeated in each visitor (a three-line loop), but each visitor controls order, context and state. A separate walker would remove the repetition, but then depth and paths would have to be passed to visitors.
- Extension boundary (R8): StatisticsVisitor was added as a new class and a new commit; FileNode, DirectoryNode and accept were not changed. A new operation = a new Visitor implementation.
- Adding a new element type (for example SymbolicLinkNode) is the hard direction: Visitor needs a new visit(SymbolicLinkNode) method, so TotalSizeVisitor, SearchVisitor, StatisticsVisitor and all test visitors must be changed, and the compiler forces this. Visitor is good when the element types are stable and operations change often; if element types change often, a polymorphic method in the elements is simpler.

7. GitHub repository

https://github.com/<your-username>/visitor-filesystem (replace with the real link)

UML class diagram

